

Deep Learning for NLP

Student name: *Angelos Dorotheos Chatzopoulos*
cs: 7115112400027

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Fall Semester 2025*

Contents

1	Abstract	3
2	Data Analysis and Preprocessing	3
2.1	Data initial analysis	3
2.1.1	Dimensions of Datasets and Checking for NULL or Duplicated Rows	3
2.1.2	Distribution of sentiment labels	4
2.1.3	Distribution of sentiment labels	4
2.1.4	Tweets length before preprocessing	4
2.1.5	Finding top 10 unigrams, bigrams & trigrams before preprocessing	5
2.1.6	Counting emails and user mentions in tweets	5
2.2	Tweets preprocessing & further analysis	6
2.2.1	Tweets length after preprocessing with stop words filtering enabled	8
2.2.2	Distribution of sentiment labels after applying preprocessing	8
2.2.3	Finding top 10 unigrams, bigrams & trigrams after applying preprocessing	9
3	TF-IDF Vectorization - Feature Extraction	10
3.1	Experimenting on TF-IDF n-gram parameter	12
3.1.1	Experimenting on TF-IDF n-gram range hyperparameter with NPR datasets	12
3.1.2	Experimenting on TF-IDF n-gram range hyperparameter with PRFS datasets	12
3.1.3	Experimenting on TF-IDF n-gram range hyperparameter with PRPS datasets	12
3.2	Experimenting on TF-IDF max features hyperparameter with PRPS datasets	15
3.3	Experimenting on TF-IDF max df hyperparameter with PRPS datasets	15
4	Logistic Regression Development and Evaluation	17
4.1	Logistic Regression LBFGS solver	18
4.2	Logistic Regression LibLinear solver	18
4.3	Logistic Regression Newton-CG solver	18
4.4	Logistic Regression SAG solver	20
4.5	Logistic Regression SAGA solver	21

5	Results and Overall Analysis	22
5.1	Cross-Validation Experiments	22
5.2	Best Model Performance Evaluation	23
6	Bibliography	25

1. Abstract

This project develops a sentiment analysis classifier using Logistic Regression model and TF-IDF feature extraction on a English-language Twitter dataset. The datasets consist of three features: ID, text, and label where label represents positive sentiment (1) or negative sentiment (0) for each tweet. An exploratory data analysis (EDA) is performed on a train set and a validation set to understand the distribution of labels and characteristics of text. Next, based on the data analysis, text preprocessing is applied to remove any unuseful parts of the text, and then it is converted into TF-IDF vectors where multiple hyperparameter experiments were conducted. Moreover, a Logistic Regression classifier is implemented and experiments were conducted to tune the hyperparameters of the model using different solver algorithms and cross-validation techniques. Finally, by using multiple performance metrics such as accuracy, precision, recall, and F1-score a final model is selected.

2. Data Analysis and Preprocessing

2.1. Data initial analysis

Initially, an in-depth analysis is performed to find the key characteristics of the given train and validation datasets [6]. This exploratory data analysis (EDA) allowed us to be aware of the distribution of sentiments labels, tweets length, username and email usage and most common tokens within our datasets using histograms and word clouds.

2.1.1. Dimensions of Datasets and Checking for NULL or Duplicated Rows. In this step, we examine the dimensions of the given datasets. Table 1 shows the dimensions of train validation and test datasets. Training and validation datasets contain three columns (ID, Text, Label) while test dataset contains only 2 columns (ID, Text).

Dataset	Dimensions
Train	(148388, 3)
Validation	(42396, 3)
Test	(21199, 2)

Table 1: Dimensions of Datasets

Furthermore, null values and duplicate rows are examined. Observing Table 2, we notice that there are no missing or duplicate values in both training and validation datasets.

Checks	Train Set	Validation Set
Missing Values in ID	0	0
Missing Values in Text	0	0
Missing Values in Label	0	0
Duplicate Rows	0	0

Table 2: Missing Values and Duplicate Rows Check

2.1.2. Distribution of sentiment labels. In this step, the distribution of sentiment labels are examined to check for any dataset imbalance. Figure 1 shows that both training and validation datasets have equal percentage of positive and negative sentiment labels.

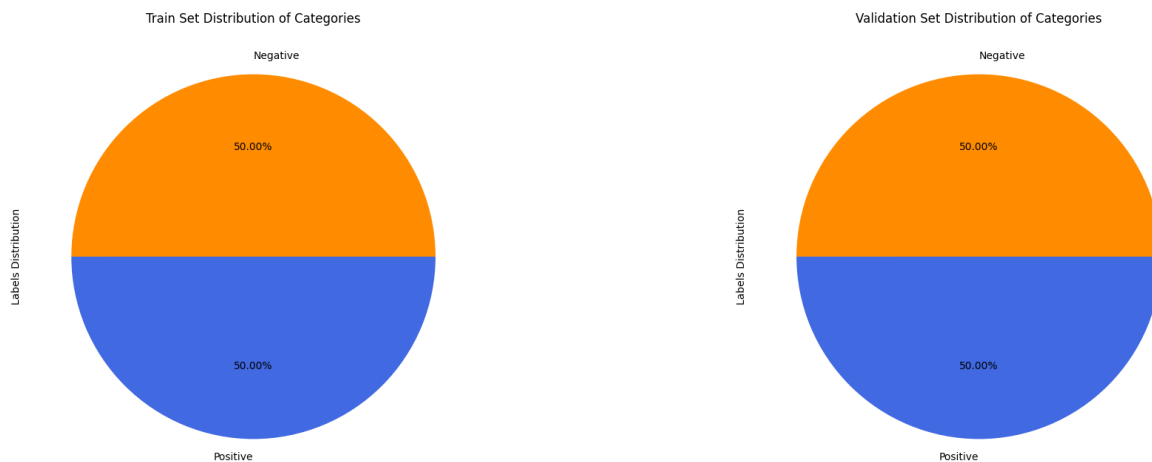


Figure 1: Distribution of tweet sentiment labels in the Training (left) and Validation (right) datasets.

2.1.3. Distribution of sentiment labels. Furthermore, we show the most common words before applying any preprocessing in training dataset using word clouds [1]. Figure 2, shows the overall distribution of words using the entire training dataset. The second figure, Figure 3, presents two separate word clouds: one for tweets with positive sentiment and another for tweets with negative sentiment. These plots help visualize the most common keywords associated with each sentiment label. Some non-English characters were found and will be removed later during the tweets preprocessing phase of the datasets.

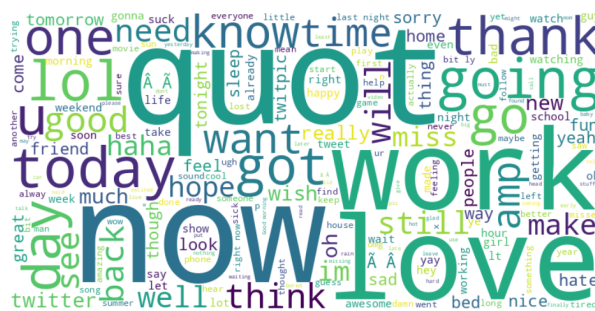


Figure 2: Word cloud using entire training dataset.

After examining the word clouds we notice that the most common keywords in positive tweets are "love", "thank", "good". In contrast, tweets with negative sentiment analysis highlight words such as "work", "now", "today".

2.1.4. Tweets length before preprocessing. Observing Figure 4 we notice that most tweets contain between 5 - 15 words with an average of 13.29 words per text. Moreover,



Figure 3: Word cloud representations of training set tweets: Positive sentiment (left) and Negative sentiment (right).

Figure 5 shows the minimum and maximum length of tweets in training dataset before applying any preprocessing step.

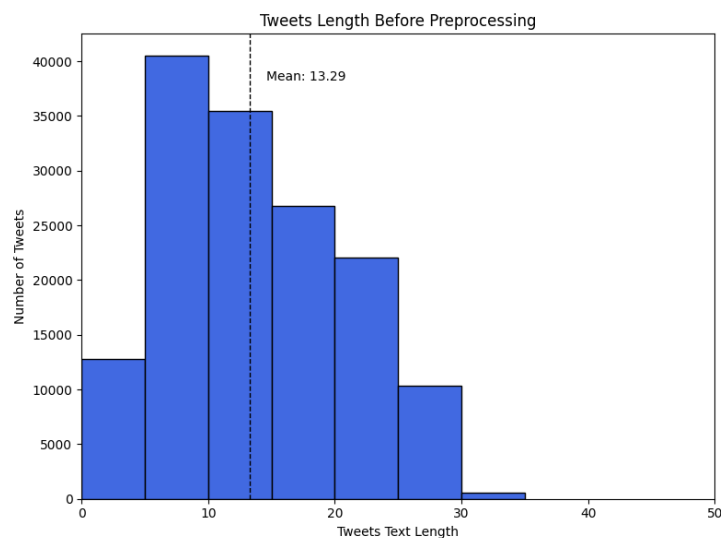


Figure 4: Distribution of tweet lengths in the dataset.

We preserve all the tweets even if the text length is one (but not white space character tweets in train and validation sets).

2.1.5. Finding top 10 unigrams, bigrams & trigrams before preprocessing. In this step, we analyze the most frequent unigrams, bigrams, and trigrams from the dataset before applying any preprocessing. These n-grams provide insight into the most common word combinations and phrases used in tweets. Figures 6 and 7 show that most unigrams and bigram tokens are common stop words that do not contribute in our sentiment analysis.

2.1.6. Counting emails and user mentions in tweets. Figure 8 shows that a significant number of user mentions are consisted in tweets while the frequency of emails is low. During preprocessing these tokens will be anonymized for privacy reasons and a cleaner dataset.

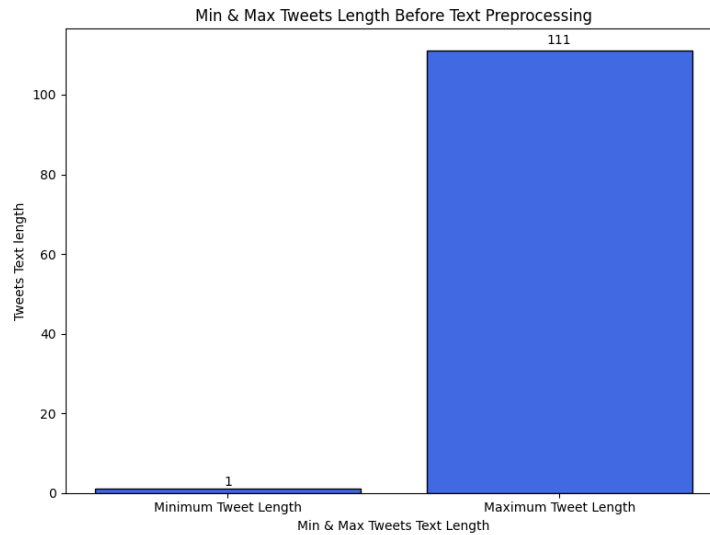


Figure 5: Minimum and maximum length of tweets in training dataset before applying text preprocessing.

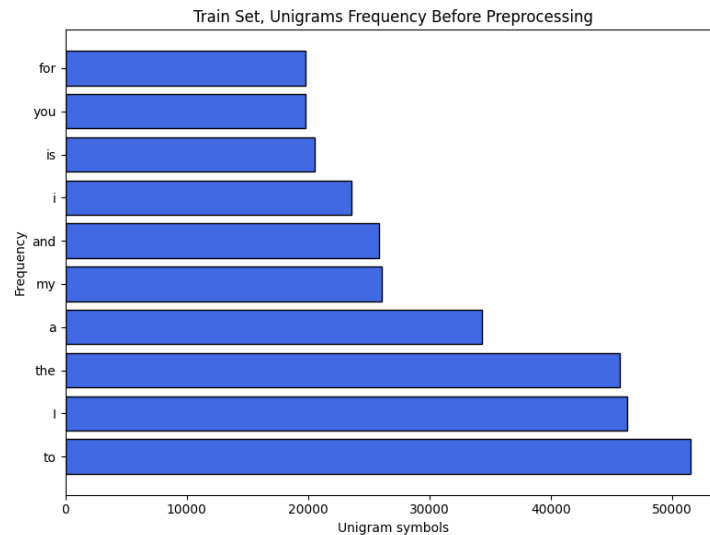


Figure 6: Top 10 unigrams in training set before applying text preprocessing.

2.2. Tweets preprocessing & further analysis

Text preprocessing is a fundamental technique in natural language processing (NLP) that aims to clean the text by running a series of transformations such as lowercase, eliminate and convert tokens. These steps reduce noise and help the model focus on the most important tokens and improve the efficiency of the model improving the model's accuracy and generalization. Tweets preprocessing experiments are performed in two parts:

- Preprocessing with stop words filtering enabled (where we perform further EDA).
- Preprocessing with stop words preservation enabled (where we notice later better accuracy scores by preserving them).

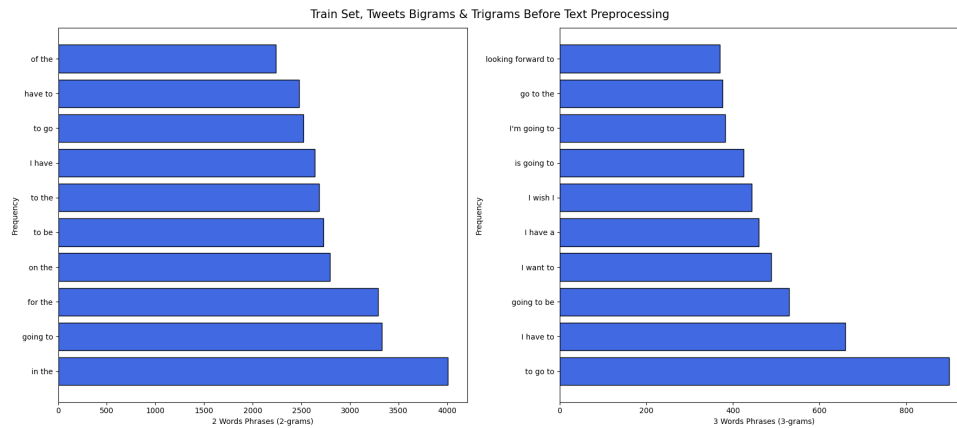


Figure 7: Top 10 bigrams and trigrams in training set before applying text preprocessing.

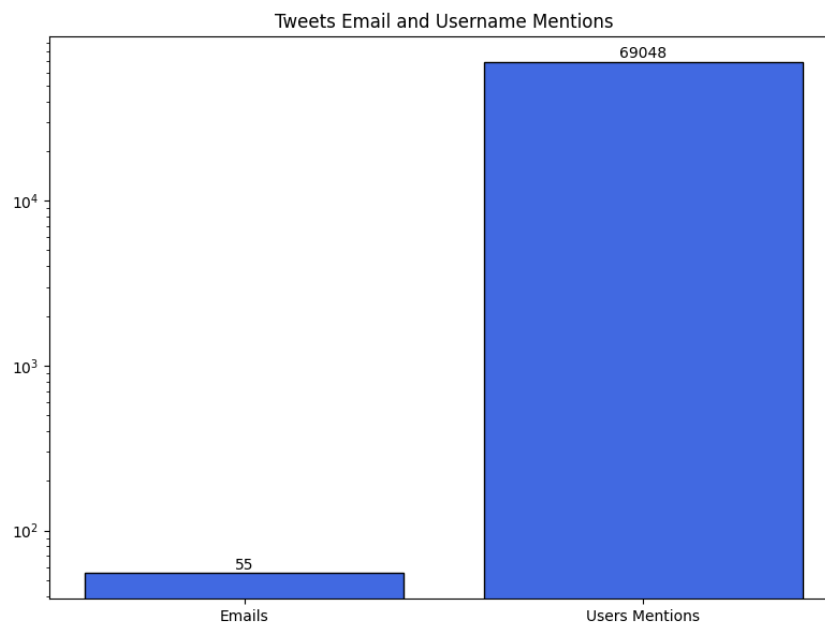


Figure 8: Counting email and user mentions in tweets before preprocessing on training dataset.

A comprehensive pre-process approach is applied on tweets [5, 2, 4, 3]. The preprocessing steps applied on all the datasets (train, validation, test) are the following:

- **Removing Non-ASCII Characters:** Non-ASCII characters are removed from tweets.
- **Lowercasing:** All characters in the text are converted to lowercase.
- **Expanding Contractions:** Shortened words are expanded to their full forms without apostrophes (e.g., "didn't" becomes "did not").
- **Removing Punctuation:** All punctuation, except for '@' (used for user mentions and email addresses), is removed from the text.
- **Anonymizing Mentions:** User mentions (e.g., @john) are replaced with '@user' to maintain anonymity.

- **Anonymizing Email Addresses:** Email addresses are replaced with 'XXX@email.com' to maintain anonymity.
- **Removing URLs:** Any URLs present in the text are removed.
- **Removing Digits:** All numeric characters are removed from the text.
- **Filtering Stop Words:** If stop word filtering is enabled, common stop words are removed from the text.
- **Lemmatization:** Words are converted to their root form (e.g., "studying" becomes "study").
- **Removing Empty Rows:** Any row with zero text length is removed (applied only to the training and validation sets).

2.2.1. Tweets length after preprocessing with stop words filtering enabled. Due to the removal of stop words, the length of tweets has decreased. Observing Figure 9, we notice that most tweets contain between 5 - 10 words with a mean value of 7.35. Furthermore, Figure 10 shows that the minimum and maximum length of tweets in training dataset after applying preprocessing. We notice that the maximum length of tweets has decreased from 111 to 28.

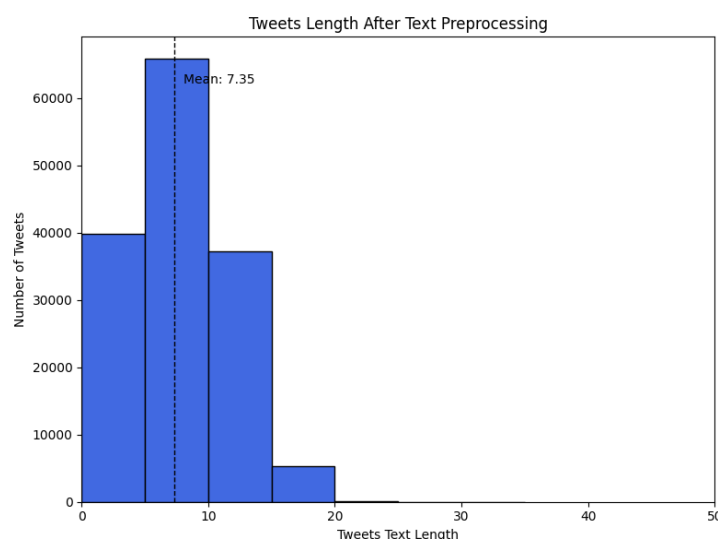


Figure 9: Distribution of tweet lengths in the dataset after applying preprocessing with filtering stop words enabled.

2.2.2. Distribution of sentiment labels after applying preprocessing. We show the most common words in the training dataset using wordclouds after running preprocessing with stop words filtering enabled. Figure 11, shows the overall distribution of words from the entire training dataset. The second figure, Figure 12, presents two separate word clouds: one for tweets with positive sentiment and another for tweets with negative sentiment.

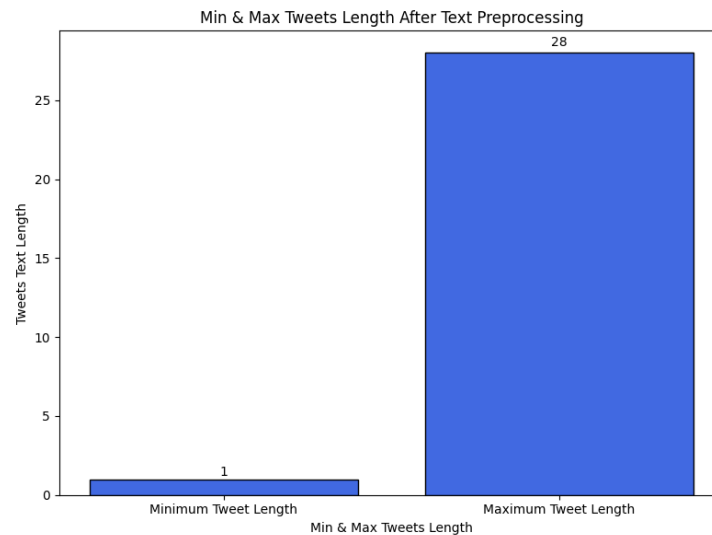


Figure 10: Minimum and maximum length of tweets in training dataset after applying text preprocessing with filtering stop words enabled.

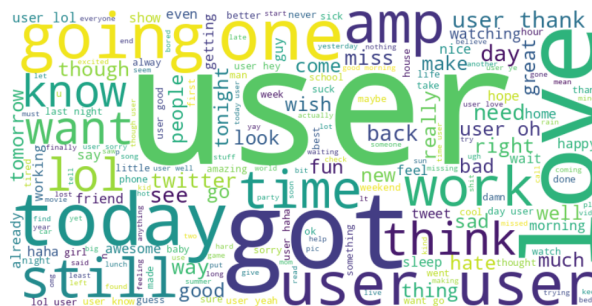


Figure 11: Word cloud using entire training dataset.



Figure 12: Word cloud representations of training set tweets after preprocessing: Positive sentiment (left) and Negative sentiment (right).

After examining the word clouds, after running preprocessing, we notice that the most common word is user name mentions overall. In addition, positive words such as "love", "thank" and "good" remain high in frequency. Finally, tweets with negative sentiment analysis highlight words such as "going", "today", "work".

2.2.3. Finding top 10 unigrams, bigrams & trigrams after applying preprocessing. Figures 13, 14, 15, 16 show a clearer representation of the most frequent words compared to the no-preprocessed plots. This is mostly due to the removal of stop words.

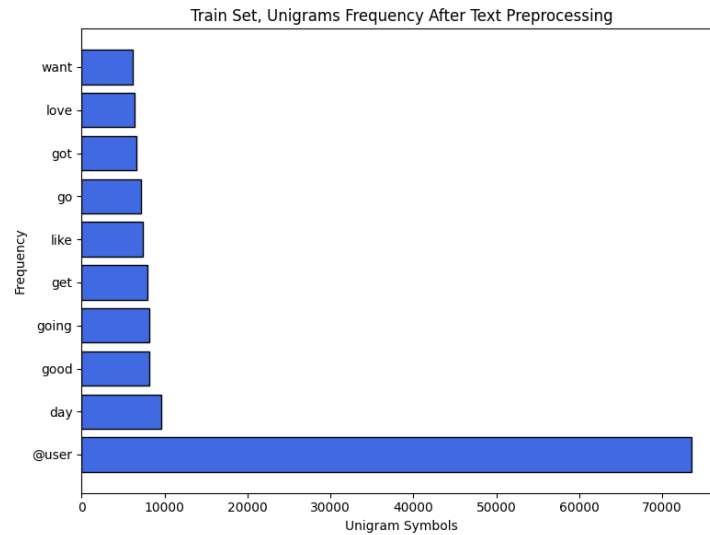


Figure 13: Top 10 unigrams in training set after applying tweets preprocessing.

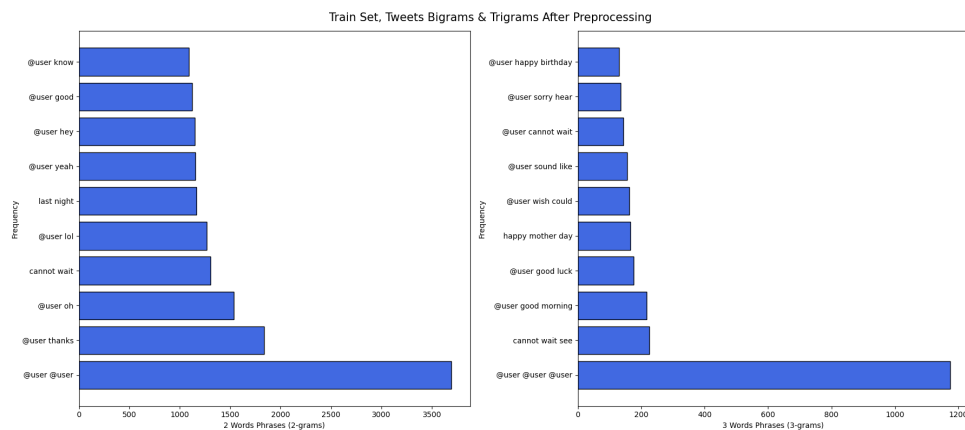


Figure 14: Top 10 bigrams and trigrams in training set after applying tweets preprocessing.

3. TF-IDF Vectorization - Feature Extraction

TF-IDF is a feature extraction technique that transforms the text (words or tokens) into a numerical representation based on the importance of tokens relative to the entire dataset [9]. Before continuing with Logistic Regression hyperparameters tuning and optimizations, multiple experiments were conducted on various TF-IDF hyperparameters while keeping Logistic Regression model [10] with the default configuration. This allowed us to experiment with multiple parameters on TF-IDF and obtain the optimal configuration for the given dataset. TF-IDF experimentation is conducted in three dataset configurations: non-preprocessed dataset, then on pre-processed with filtered stop words, and finally on pre-processed dataset with preserved stop words (where we notice better accuracy results). The model training is done using the train set while the validation set is used for performance metrics evaluation. Learning curve is calculated using the corresponding train set (preprocessed or non-preprocessed based on the configuration).

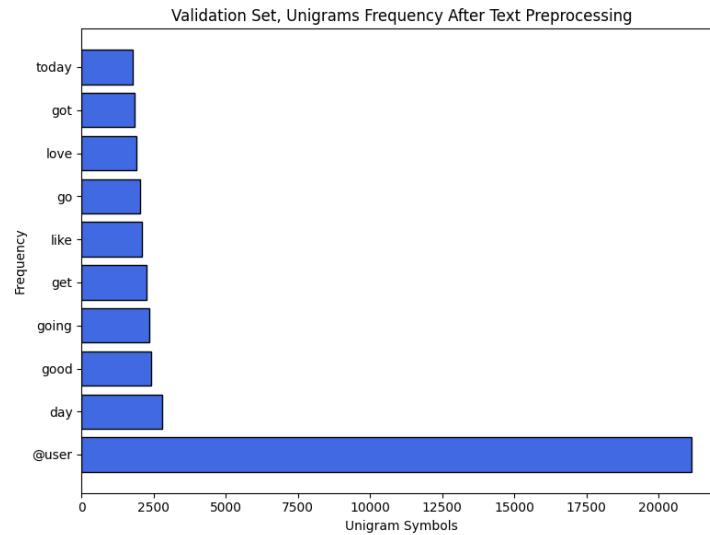


Figure 15: Top 10 unigrams in validation set after applying tweets preprocessing.

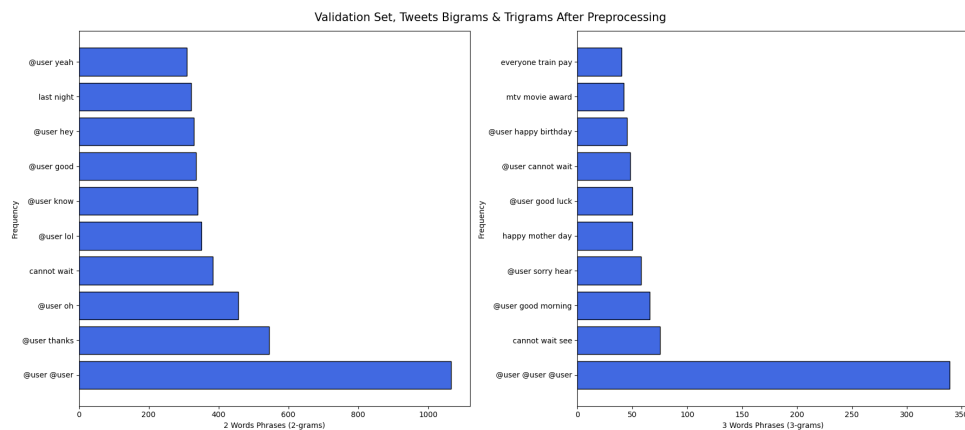


Figure 16: Top 10 bigrams and trigrams in validation set after applying tweets preprocessing.

During the experiments, the following acronyms are used on the configuration labels for datasets.

- **NPR**: No Preprocessing
- **PRFS**: Preprocessing & Filtering Stopwords
- **PRPS**: Preprocessing & Preserving Stopwords

The performance evaluation metrics used on validation set are the following.

- **Accuracy**: Measures the proportion (ratio) of correctly classified instances to the total instances.
- **Precision**: Measures the accuracy of positive predictions.
- **Recall**: Measures the proportion of actual positives that were actually predicted correctly as positives.

- **F1 Score:** Balances precision and recall and it is mainly used for imbalanced datasets detection.

3.1. Experimenting on TF-IDF n-gram parameter

In this section, we experimented with TF-IDF n-gram hyperparameter which is used to extract different lengths of n-grams. For example, (1, 1) (default value) means that only unigrams will be captured, while (1, 2) means both unigrams and bigrams will be stored, etc. These experiments were carried out on NPR, PRFS & PRPS datasets to examine whether the preprocessing steps affect performance metrics. TF-IDF hyperparameters are selected based on highest accuracy score of the model performed.

3.1.1. Experimenting on TF-IDF n-gram range hyperparameter with NPR datasets. In this step, we tested n-gram hyperparameters (1, 1), (1, 2), (2, 2), and (2, 3) on a no-pre-processed (NPR) datasets on default Logistic Regression parameters in order to find the optimal n-gram range value.

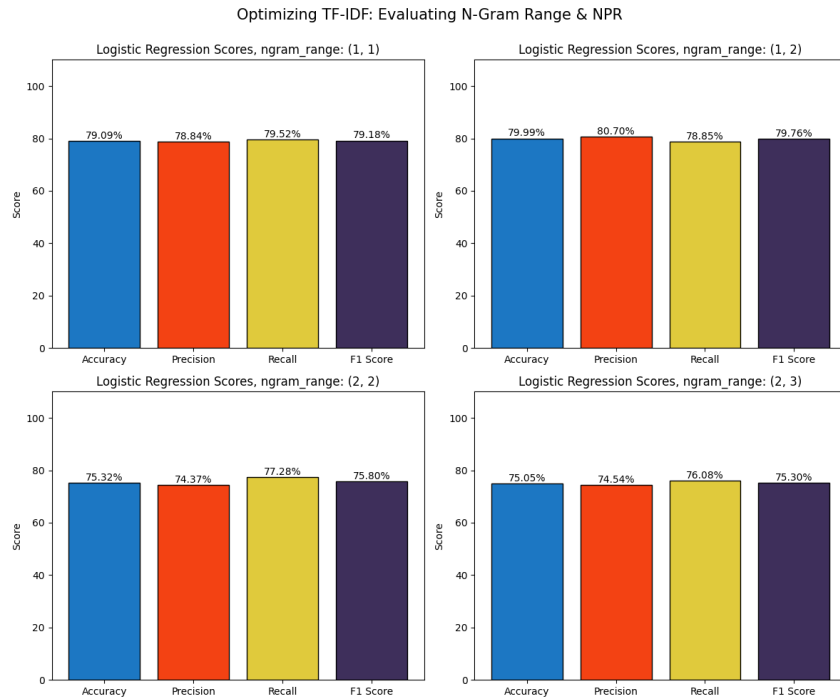


Figure 17: N-gram range experimentation on NPR datasets.

3.1.2. Experimenting on TF-IDF n-gram range hyperparameter with PRFS datasets. In this step, we tested n-gram hyperparameters (1, 1), (1, 2), (2, 2), and (2, 3) on pre-processing with filtered stop words (PRFS) datasets on default Logistic Regression parameters in order to find the optimal n-gram range value.

3.1.3. Experimenting on TF-IDF n-gram range hyperparameter with PRPS datasets. In this step, we tested n-gram hyperparameters (1, 1), (1, 2), (2, 2), and (2, 3) on pre-processing with preserved stop words (PRPS) datasets on default Logistic Regression parameters in order to find the optimal n-gram range value.

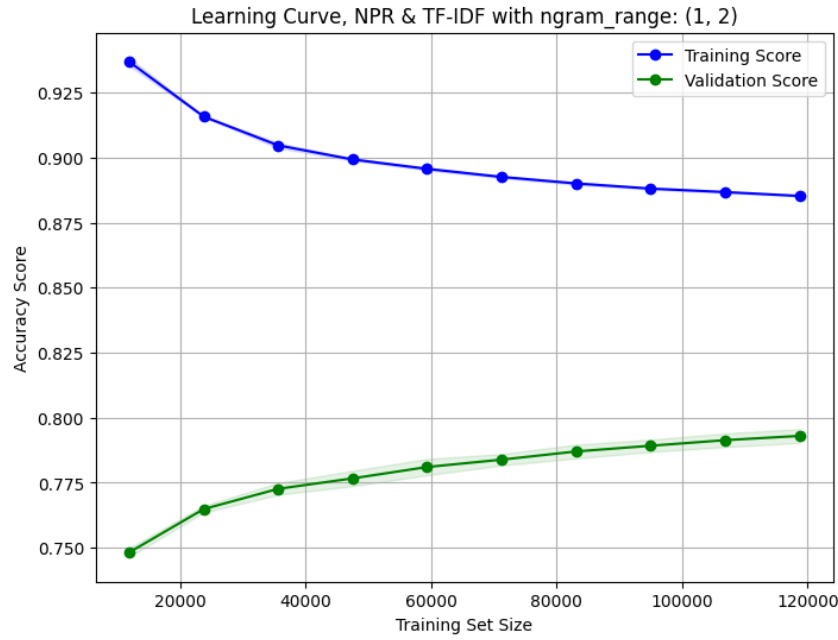


Figure 18: Learning curve on n-gram range: (1, 2) on NPR datasets.

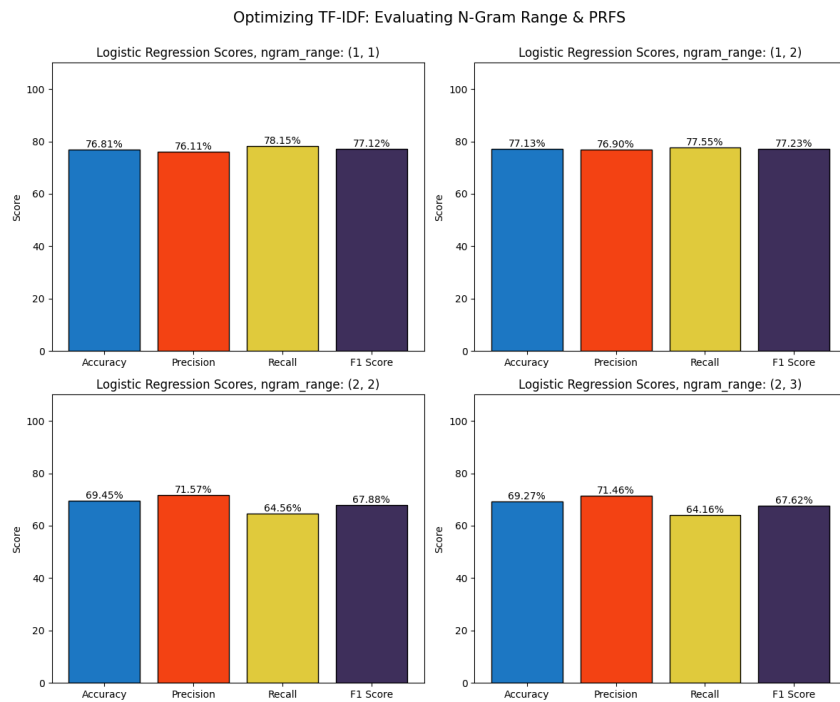


Figure 19: N-gram range experimentation on PRFS datasets.

Figures 17, 19, 21 show the performance metrics for accuracy, precision, recall, and F1 score of the Logistic Regression model. We notice that all dataset configurations have very similar performance metrics. The best model is selected based on the highest accuracy score, therefore **the most optimized configuration thus far is n-gram range (1, 2) using PRPS datasets**. Perhaps, some stops words may be important for tweets sentiment analysis and increase slightly the accuracy of our model. Moreover, Figures 18, 20, 22 show the learning curves [14] for each dataset configuration with the highest

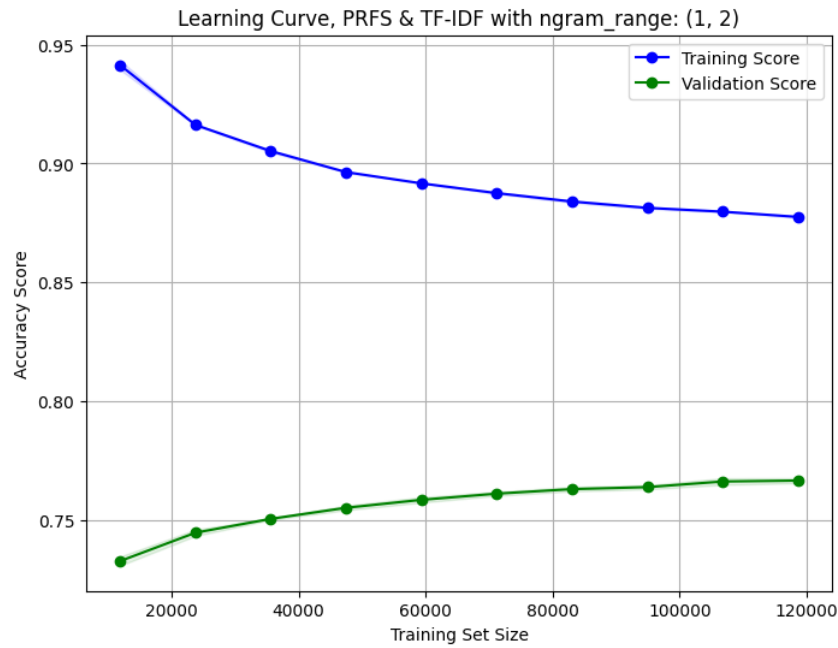


Figure 20: Learning curve on n-gram range: (1, 2) on PRFS datasets.

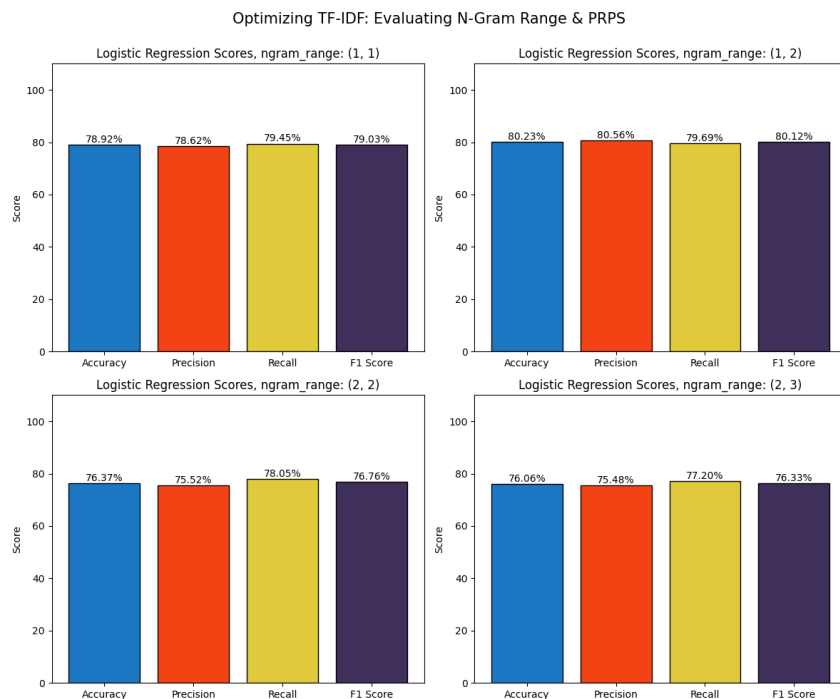


Figure 21: N-gram range experimentation on PRPS datasets.

accuracy score (n-gram range (1,2)). We notice that the Logistic Regression model overfits due to the large gap between the training and validation set accuracy, which will be fixed by further experimenting on TF-IDF and Logistic Regression hyperparameters.

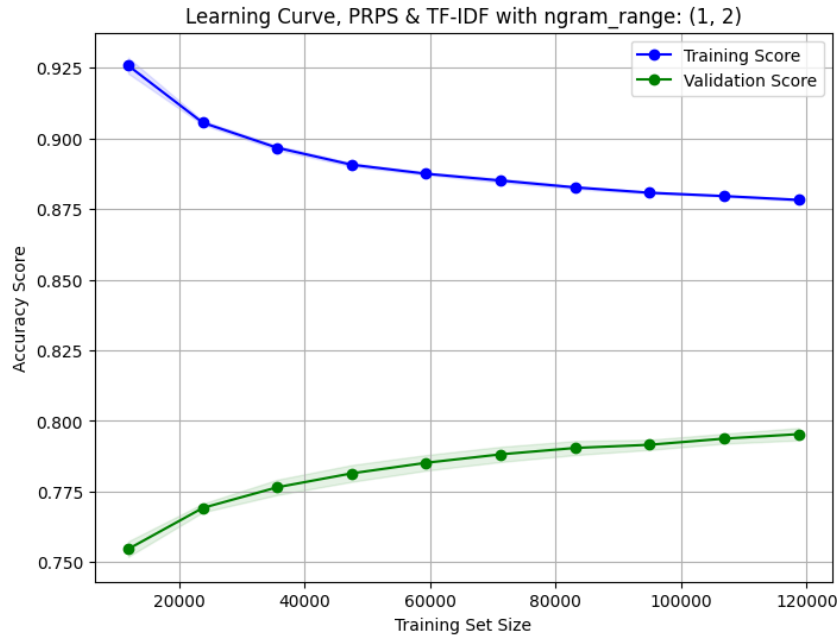


Figure 22: Learning curve on n-gram range: (1, 2) on PRPS datasets.

3.2. Experimenting on TF-IDF max features hyperparameter with PRPS datasets

In this section, we experimented with TF-IDF max features hyperparameter. We tested having 300K, 400K, 500K & 600K TF-IDF features and examine whether the dimensionality reduction affects the performance metrics of the model.

Since the n-gram range (1, 2) with PRPS datasets is the best configuration so far, this dataset input & TF-IDF parameter are kept constant throughout the following experiments.

Figure 23, shows the performance metrics for accuracy, precision, recall and F1 score of the model. We notice that all configurations have very similar performance metrics and that the accuracy is also very similar with all other performance metrics. The most optimized model is selected based on the highest accuracy score. Furthermore, Figure 24, show the learning curves using the training dataset with the best configuration (max features: 300K, n-gram range (1,2)). We notice that the model has high training performance, with low validation set performance, and thus it still overfits.

3.3. Experimenting on TF-IDF max df hyperparameter with PRPS datasets

In this section, we experimented with the max df hyperparameter for TF-IDF, which ignores tokens that have higher frequency than this threshold. We tested various values for the max df, including 0.98, 0.96, 0.94 and 0.92 and examine how changes in this hyperparameter affect the performance metrics of the model. Again, the default hyperparameters are used on Logistic Regression model.

Since the n-gram range (1, 2) and max features set to 300K are the best hyperparameters so far (on PRPS datasets), both parameters are preserved throughout the following experiments.

Observing Figure 25, we notice that max df hyperparameter is not affecting the model's performance metrics, thus **we do not make any change on this hyperparam-**

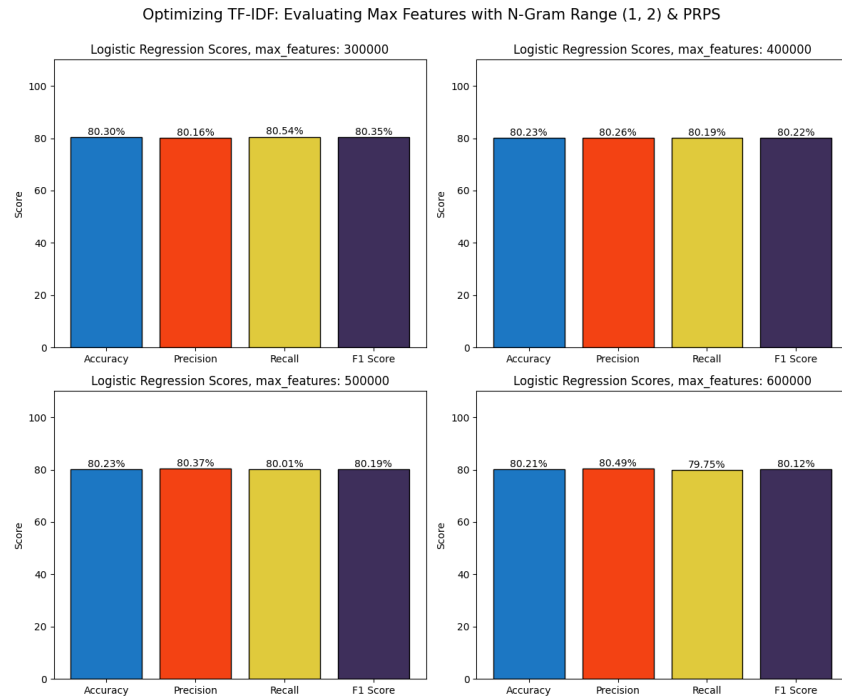


Figure 23: Max features hyperparameter experimentation on PRPS datasets.

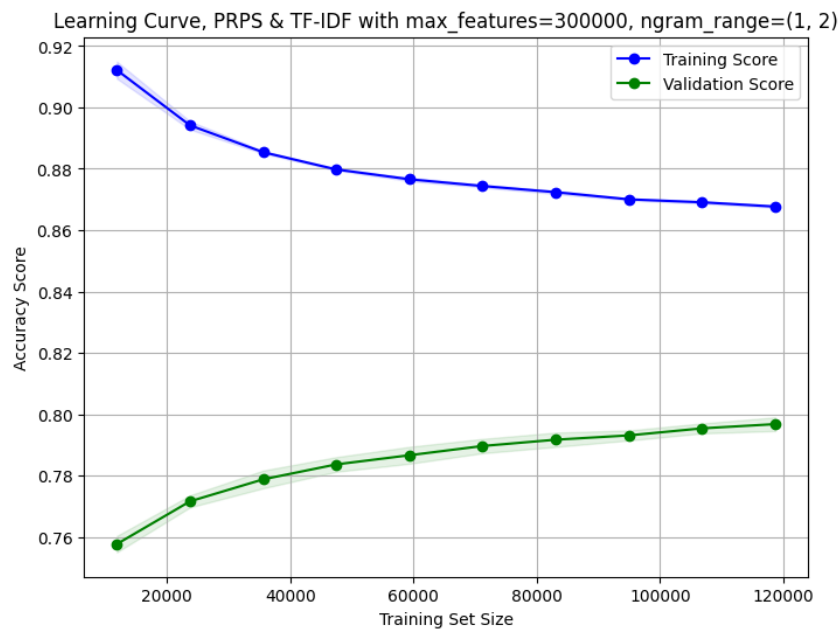


Figure 24: Learning curve on best max features parameter: 300K on PRPS datasets.

eter of TF-IDF.

Table 3 shows the best TF-IDF configuration that will be used on Logistic Regression experiments with PRPS datasets.

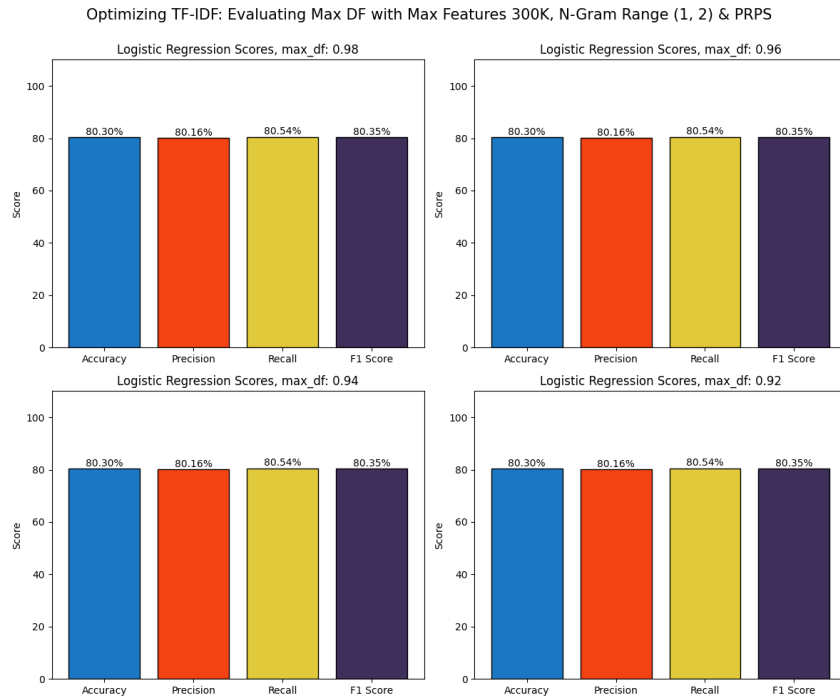


Figure 25: Max df hyperparameter experimentation on PRPS datasets.

Parameter	Value
ngram_range	(1,2)
max_features	300K

Table 3: TF-IDF Parameters

4. Logistic Regression Development and Evaluation

In this section, multiple solvers for Logistic Regression were tested, each supporting different regularization methods (penalties). The plan is to evaluate the performance of the model using different algorithms and regularization methods while evaluating the learning curve to check whether there is underfitting or overfitting in each model configuration. Table 4 shows the solvers and the penalties they support that were used during the evaluation. The model training is done using the train set while the validation set is used for performance metrics evaluation. **Note that newton-cholesky solver was not evaluated because it exceeds Kaggle's memory limit.**

Solver	Penalty Type
lbfgs	L2 penalty
liblinear	L1 & L2 penalty
newton-cg	L2 penalty
sag	L2 penalty
saga	Elasticnet & L1 & L2 penalty

Table 4: Logistic Regression solvers and their supported penalties.

4.1. Logistic Regression LBFGS solver

Evaluating the performance of LBFGS solver (Figure 26) on the validation set using the default values in the remaining hyperparameters and evaluating the learning curve of the model using the training dataset. Observing Figure 27, we notice that the model overfits.

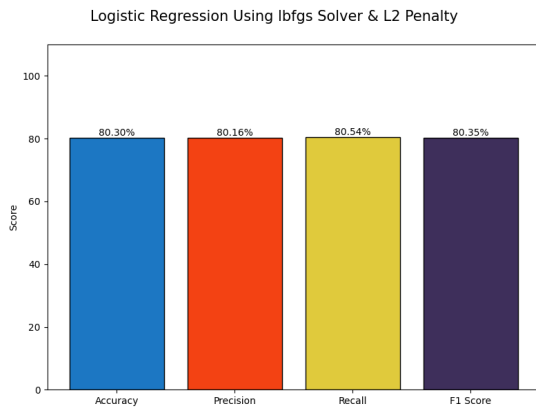


Figure 26: LBFGS Logistic Regression using L2 penalty on PRPS.

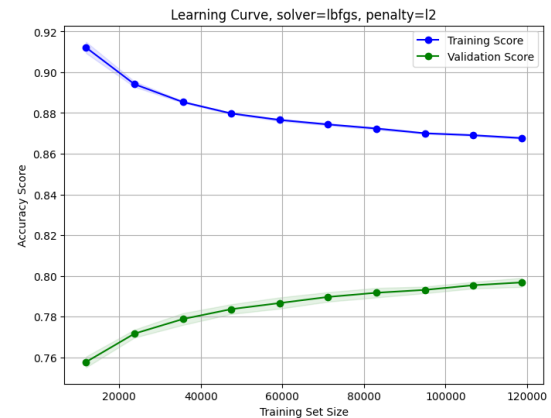


Figure 27: Learning curve on LBFGS Logistic Regression using L2 penalty on PRPS.

4.2. Logistic Regression LibLinear solver

Evaluating the performance of LibLinear solver (Figure 28) on the validation set using the default values for the remaining hyperparameters and evaluating the learning curve of the model using the training dataset. Observing Figure 29, we notice that the L1 penalty model has balanced performance on both the training and validation sets. In contrast, Figure 30 shows that the training and validation sets do not converge, resulting in an overfitting model.

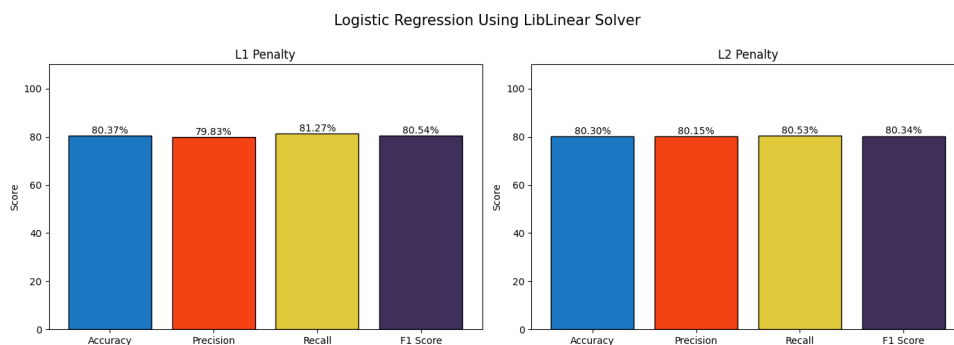


Figure 28: LibLinear Logistic Regression using L2 penalty on PRPS.

4.3. Logistic Regression Newton-CG solver

Evaluating the performance of Newton-CG solver (Figure 31) on the validation set

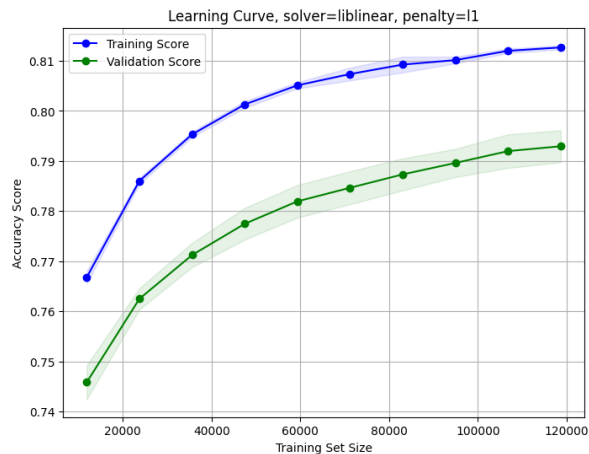


Figure 29: Learning curve on LibLinear Logistic Regression using L1 penalty on PRPS.

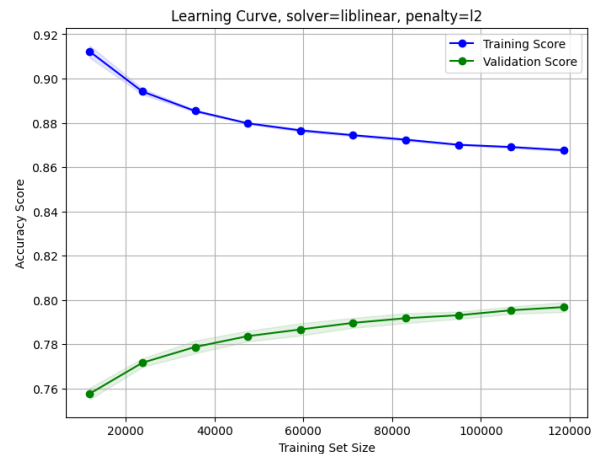


Figure 30: Learning curve on LibLinear Logistic Regression using L2 penalty on PRPS.

using the default values in the remaining hyperparameters and evaluating the learning curve of the model using the training dataset. Observing Figure 32, we notice that the model overfits.

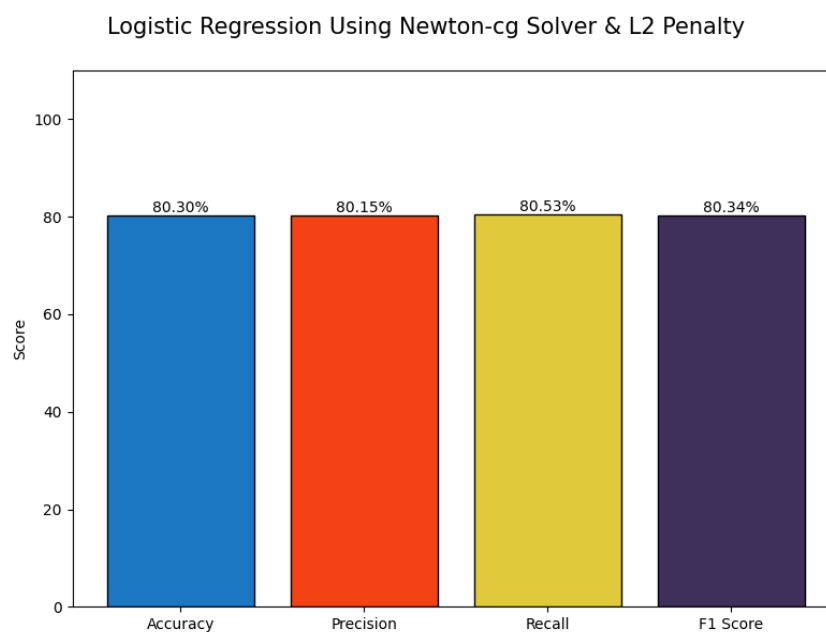


Figure 31: Newton-CG Logistic Regression using L2 penalty on PRPS.

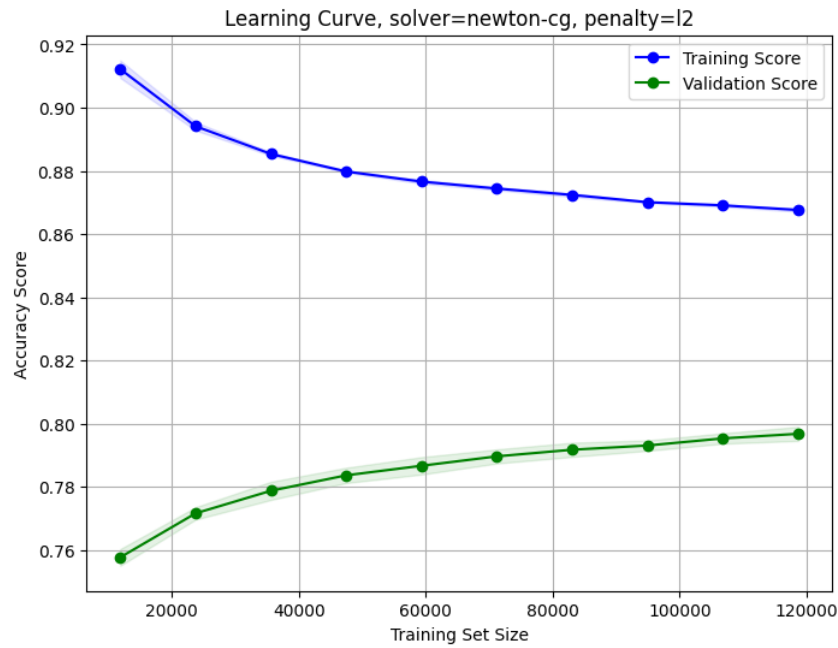


Figure 32: Learning curve on Newton-CG Logistic Regression using L2 penalty on PRPS.

4.4. Logistic Regression SAG solver

Evaluating the performance of SAG solver (Figure 33) on the validation set using the default values in the remaining hyperparameters and evaluating the learning curve of the model using the training dataset. Observing Figure 34, we notice that the model overfits.

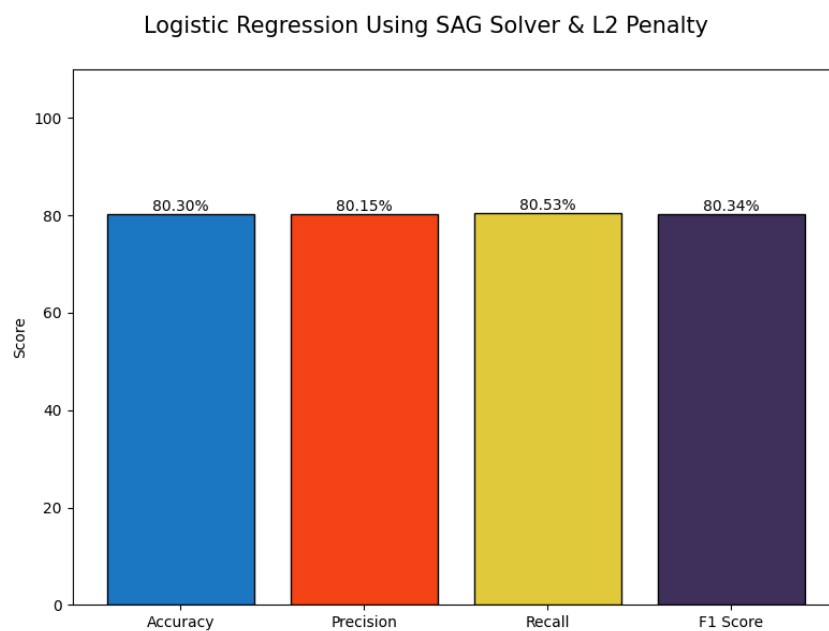


Figure 33: SAG Logistic Regression using L2 penalty on PRPS.

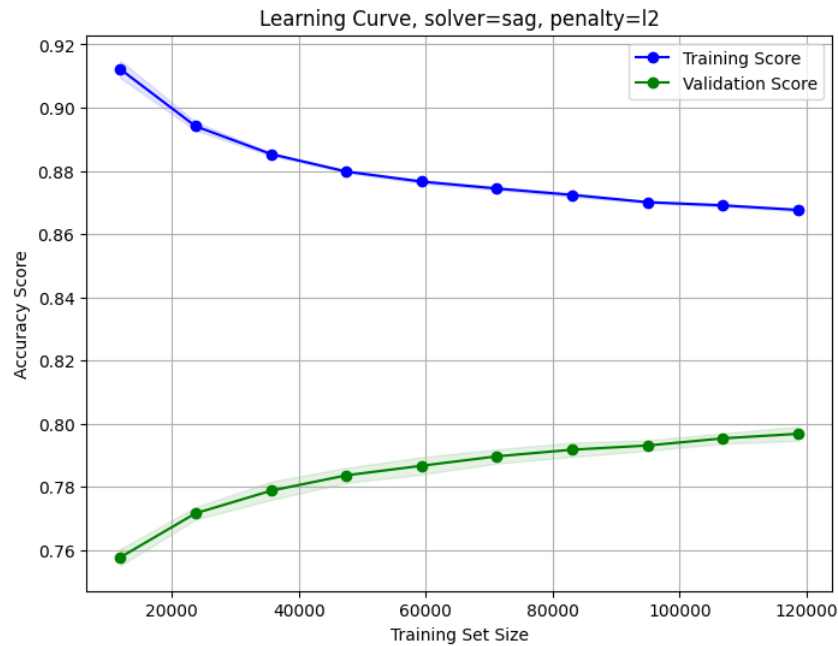


Figure 34: Learning curve on SAG Logistic Regression using L2 penalty on PRPS.

4.5. Logistic Regression SAGA solver

Evaluating the performance of SAGA solver (Figure 35) on the validation set using the default values in the remaining hyperparameters and evaluating the learning curve of the model using the training dataset. Notice that SAGA solver supports L1, L2 & Elasticnet penalties. Observing Figure 36, we notice that the SAGA solver using L1 and Elasticnet penalties does not overfit or underfit. In contrast, the SAGA solver using the L2 penalty overfits.

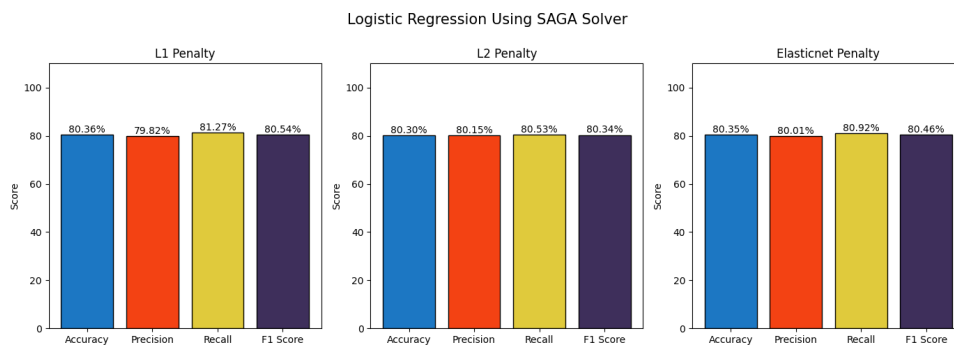


Figure 35: Learning curve on Logistic Regression on PRPS.

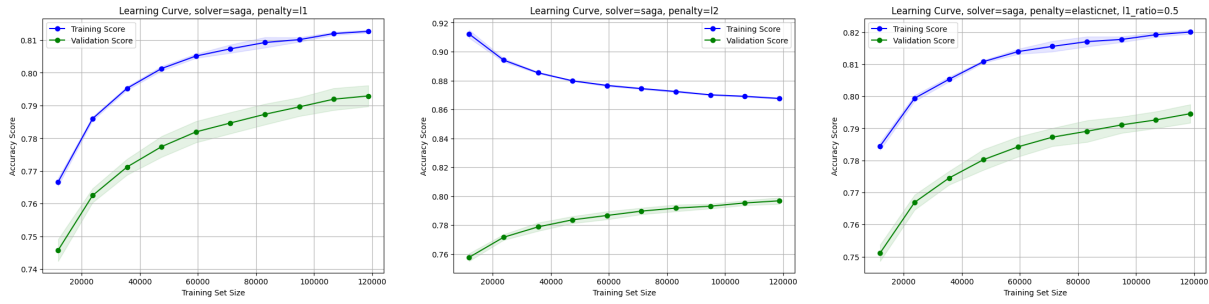


Figure 36: Learning curves on SAGA Logistic Regression using L1 penalty (left), L2 penalty (center), and ElasticNet penalty (right) on PRPS.

5. Results and Overall Analysis

In this section, based on the previous analysis in Section 4, we run cross-validation to experiment with the hyperparameters: inverse of regularization strength ‘C’, max iterations, and L1 ratio (for Elasticnet) using the Logistic Regression solvers mentioned earlier [13, 8]. We exclude configurations where ‘C=1’ (the default Logistic Regression value) caused overfitting issues. The goal is to increase the model’s generalization ability and avoid underfitting or overfitting issues. We experiment with each configuration using 100, 200, and 300 maximum iterations, and search for the L1 ratio in Elasticnet penalty in the range of 0.3 to 0.6.

5.1. Cross-Validation Experiments

Multiple grid search (GridSearchCV) with 5-Fold cross-validation experiments were conducted on the training dataset using all the Logistic Regression solvers to achieve a more generalized model [7]. **For each grid search configuration set pool, the most optimized cross-val model is evaluated using the validation set.** Table 5 shows all the grid search configuration sets used to find an optimized model that does not have any overfit or underfit issues. Table 6 shows for each grid search hyperparameters tuning, the most optimized model. The best cross-validation model is chosen based on the highest accuracy score on validation set, thus Table 7 shows the most optimized cross-validation model chosen compared to all grid search experiments.

Solver	Penalty	C Values	Max Iter	l1_ratio
lbfgs	l2	0.001, 0.01, 0.1	100, 200, 300	-
liblinear	l1	0.001, 0.01, 0.1, 1	100, 200, 300	-
liblinear	l2	0.001, 0.01, 0.1	100, 200, 300	-
newton-cg	l2	0.001, 0.01, 0.1	100, 200, 300	-
sag	l2	0.001, 0.01, 0.1	100, 200, 300	-
saga	l1	0.001, 0.01, 0.1, 1	100, 200, 300	-
saga	l2	0.001, 0.01, 0.1	100, 200, 300	-
saga	elasticnet	0.001, 0.01, 0.1, 1	100, 200, 300	0.3, 0.4, 0.5, 0.6

Table 5: Parameter grid configurations for different solvers and penalties.

C	max_iter	penalty	solver	Accuracy	Precision	Recall	F1 Score	l1_ratio
0.1	100	l2	lbfgs	78.052%	77.949%	78.24%	78.094%	-
1.0	100	l1	liblinear	80.366%	79.829%	81.268%	80.542%	-
0.1	100	l2	liblinear	78.05%	77.945%	78.24%	78.092%	-
0.1	100	l2	newton-cg	78.05%	77.945%	78.24%	78.092%	-
0.1	100	l2	sag	78.05%	77.945%	78.24%	78.092%	-
1.0	100	l1	saga	80.361%	79.822%	81.268%	80.539%	-
0.1	100	l2	saga	78.05%	77.945%	78.24%	78.092%	-
1.0	100	elasticnet	saga	80.305%	80.062%	80.711%	80.385%	0.3

Table 6: Comparison of model parameters and performance metrics.

Parameter	Value
C	1
max_iter	100
penalty	l1
solver	liblinear

Table 7: Best cross-validation model hyperparameters

5.2. Best Model Performance Evaluation

Figure 37 shows all the performance evaluation scores using the best model from the grid search cross-validation experiments (Table 7). Notice that each performance scores are close to each other and are also above 80% (except precision). Furthermore, Figure 38 shows the learning curve of the best cross-validation model using the training set. Note that the performance in the training and validation set is balanced and also validation curve approaches the training curve without having a large gap in between, thus the final model does not have any overfitting/underfitting issues.

Validation Set Scores, Best Model, C=1, max_iter=100, penalty=l1, solver=liblinear

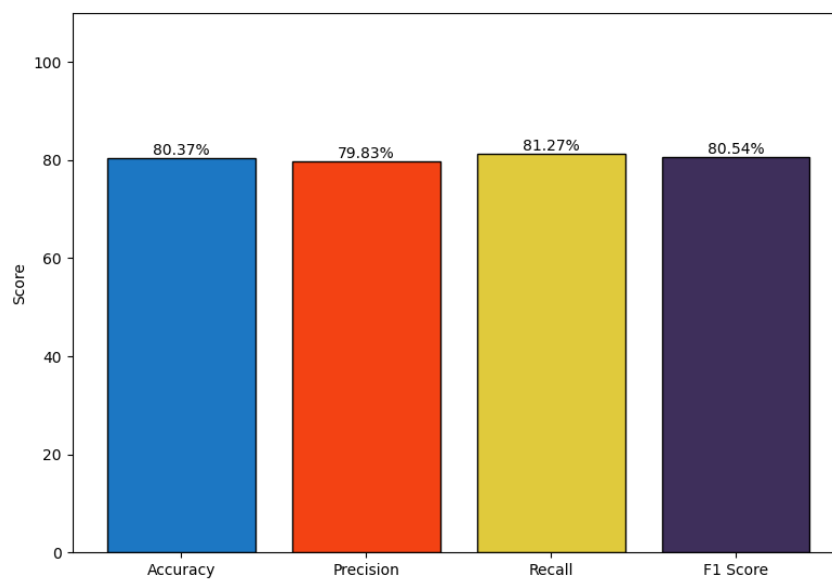


Figure 37: Best cross-validation Logistic Regression model.

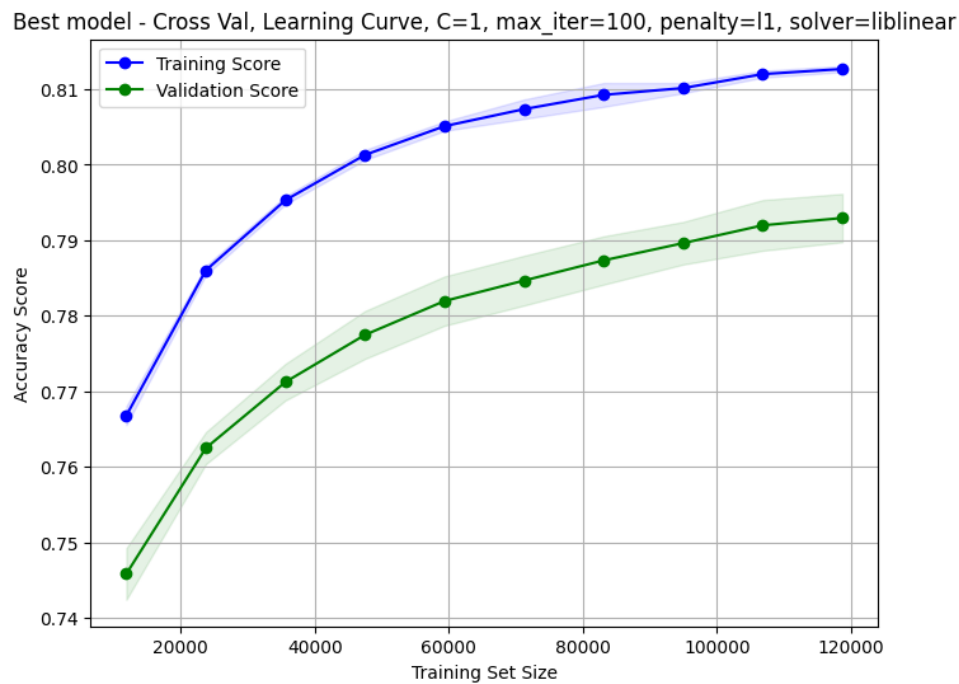


Figure 38: Learning curve on best cross-validation model.

Finally, Figure 39 shows the ROC curve [12] and confusion matrix [11] of the best cross-validation model using the validation dataset. The AUC of 0.8 shows that the model has pretty good classification performance. The confusion matrix shows us the distribution of misclassification presenting the false negative and false positive predictions.

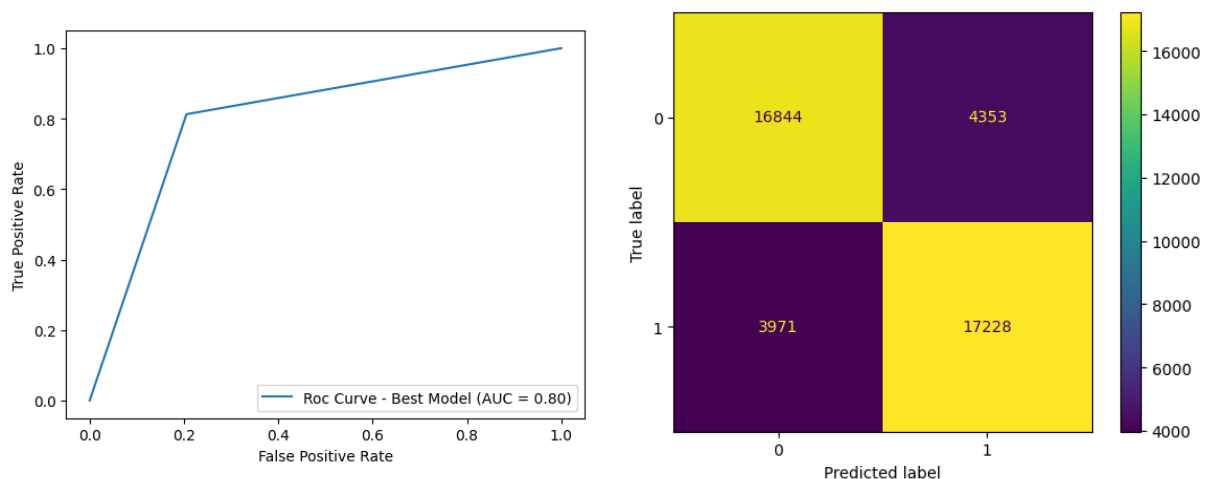


Figure 39: ROC curve & confusion matrix using the best cross-validation model.

The accuracy score on the test set (Kaggle competition) is 79.89%, which is very close to the validation set accuracy, indicating good generalization of the final Logistic Regression model.

6. Bibliography

References

- [1] amueller. Word cloud by andreas mueller. https://amueller.github.io/word_cloud/.
- [2] geeksforgeeks. Check if email address is valid or not in python. <https://www.geeksforgeeks.org/check-if-email-address-valid-or-not-in-python/>.
- [3] geeksforgeeks. Python nltk: Tweettokenizer. <https://www.geeksforgeeks.org/python-nltk-nltk-tweettokenizer/>.
- [4] geeksforgeeks. Python regex lookbehind. <https://www.geeksforgeeks.org/python-regex-lookbehind/>.
- [5] geeksforgeeks. Text preprocessing for nlp tasks. <https://www.geeksforgeeks.org/text-preprocessing-for-nlp-tasks/>.
- [6] matplotlib. matplotlib examples. <https://matplotlib.org/stable/gallery/index.html>.
- [7] medium. gridsearchcv, key concepts. <https://medium.com/@Mandeep2002/gridsearchcv-key-concepts-8f98ceb633e4>.
- [8] sklearn. Cross validation in scikit-learn. https://scikit-learn.org/stable/modules/cross_validation.html.
- [9] sklearn. sklearn feature extraction text tfidfvectorizer. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.
- [10] sklearn. sklearn linear model logisticregression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [11] sklearn. sklearn metrics confusion matrix. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html.
- [12] sklearn. sklearn metrics roc curve. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html.
- [13] sklearn. sklearn model selection gridsearchcv. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html.
- [14] sklearn. sklearn model selection learning curve. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.learning_curve.html.